COMPILER DESIGN LABORATORY • BCSE307P

</>

COSTRA

Cost-Based SQL Compiler & Query Optimizer

FINAL PROJECT REVIEW • PHASE 3: IMPLEMENTATION & TESTING

Arsh Shukla | 24BDS0113 | VIT Vellore

GitHub repository

github.com/arshshukla500newuser/COSTRA-SQL-Compiler

One query, an inspectable compiler pipeline

The goal is to show how the implementation works, not just list source files.

1 Read SQL

One SELECT with joins and aggregation

2 Compile stages

Tokens → AST → semantics → IR

3 Estimate plans

Synthetic cardinality + relative costs

4 Execute + verify

SQLite output, query plan and tests

What is implemented

Lexer, recursive-descent parser, AST, semantic analyzer, relational IR, heuristic optimizer and SQLite-backed execution.

Evidence

48 automated tests in the latest checked run, repeatable query outputs and saved benchmark summaries.

Important boundary

COSTRA ranks estimated plans. SQLite independently plans and executes the normalized SQL.

From design to a tested end-to-end prototype

Phase 1 · Design

- Problem statement and objectives
- SQL subset and architecture
- Compiler-stage mapping
- Prototype plan

Phase 2 · Core implementation

- Lexer and parser / AST
- Semantic validation
- Logical relational IR
- Cost model and candidate plans

Phase 3 · Final validation

- Integrated CLI demo
- Expanded parser / semantic tests
- Execution and SQLite plan evidence
- Benchmark, report and user guide

Review 3 focus: functional correctness · compiler integration · testing · demonstration · documentation · viva

One input, three review milestones

The menu offers three review-wise walkthroughs; each review marks progress in the same project.

REVIEW 1 / DESIGN

INPUT / PROBLEM

A SQL-query optimization problem and requirements.

SHOW

- Problem statement and objectives
- Supported SQL scope / grammar
- Architecture and compiler pipeline
- Initial prototype plan

OUTPUT

A reviewed design and clear implementation plan - not the full final execution.

REVIEW 2 / CORE MODULES

SAME SAMPLE QUERY

Passes through the core stages.

SHOW

- Tokens and AST
- Semantic checks and relational IR
- Candidate plan cost estimates
- SQLite result from the working prototype

OUTPUT

Evidence that the main modules work together and implementation progress is real.

REVIEW 3 / INTEGRATED SYSTEM

SAME QUERY, FULL PATH

One end-to-end demonstration.

SHOW

- Lowest estimated-cost plan and formula
- Normalized SQL and result rows
- SQLite EXPLAIN output and timings
- Invalid-input handling and tests
- Source, report and user guide

OUTPUT

Integrated, validated educational prototype plus final submission materials.

SAY: “Review 1 defined the design; Review 2 demonstrated core modules; Review 3 integrates the pipeline, cost comparison, execution, tests and final documentation.”

Why build COSTRA?

Problem

A SQL query hides several processing steps: tokenization, parsing, name validation, logical operators and plan estimation.

Project objective

Make these stages visible and executable so a student can trace one query from source text to validated representation and result rows.

Implemented scope

- SELECT projection, aliases and DISTINCT
- JOINs, WHERE predicates and expressions
- GROUP BY, HAVING, ORDER BY and LIMIT
- Semantic checks for table / alias / column references
- IR and synthetic plan-cost comparison
- SQLite execution and automated regression tests

The same query is used throughout the demo

It deliberately includes joins, filters, aggregation, HAVING, sorting and a row limit.

SQL SOURCE PROGRAM

```
SELECT
    d.name AS department,
    COUNT(e.id) AS employee_count,
    AVG(e.salary) AS average_salary
FROM Employee e
JOIN Department d ON e.dept_id = d.id
JOIN Assignment a ON e.id = a.employee_id
JOIN Project p ON a.project_id = p.id
WHERE e.salary > 60000 AND p.budget > 100000
GROUP BY d.name
HAVING AVG(e.salary) > 65000
ORDER BY AVG(e.salary) DESC
LIMIT 5;
```

Four relations

Employee · Department · Assignment · Project

Two filters

Salary > 60,000

Budget > 100,000

Aggregate result

Count and average salary grouped by department

When the professor asks me to enter my own SQL

Run the interactive menu, select option 4, then enter one supported SELECT statement.

OPTION 4 / INTERACTIVE MENU

python3 main.py

Choose: 4. Enter my own SELECT query

TYPE AT THE SQL> PROMPT

```
SELECT name, salary FROM Employee WHERE salary > 80000;
```

Type the entire SELECT statement on one line and finish it with a semicolon.

COSTRA runs lexer → parser/AST → semantic checks → relational IR → cost comparison → SQLite execution.

OPTIONAL / DIRECT COMMAND

```
python3 main.py --query
"SELECT name, salary FROM Employee WHERE salary
> 80000 ORDER BY salary DESC LIMIT 3;"
```

SAVED EXAMPLE RESULT

```
name      | salary
Ishita    | 91000
Arjun     | 85000
```

Explain: “COSTRA analyses and estimates the plan; SQLite independently executes normalized SQL.”

If syntax or semantics fail, the error is shown and SQL is not executed.

What does 'compilation' mean in COSTRA?

COSTRA FRONT-END + OPTIMIZATION

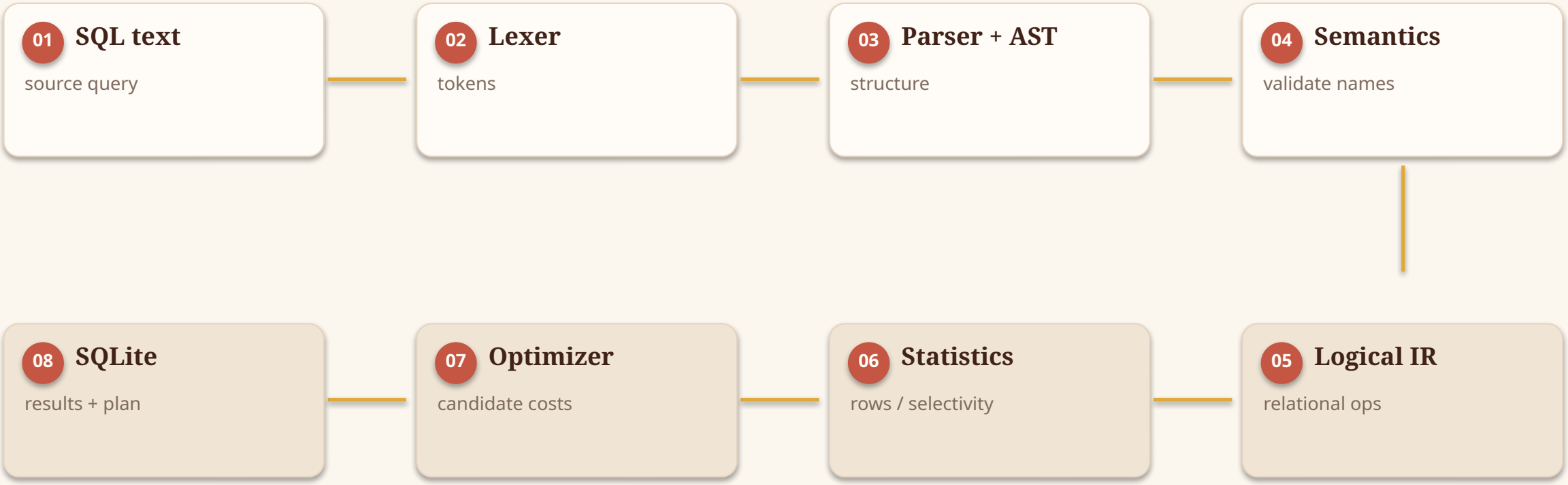
- Tokenize SQL text
- Parse tokens into an AST
- Validate table, alias and column references
- Build a relational logical plan
- Estimate candidate access paths / join orders

EXECUTION BACK-END

- Normalize the supported query
- Send it to the SQLite execution engine
- Return rows and SQLite's own query plan
- Show measured stage timings separately

COSTRA is a SQL-language educational compiler / optimizer prototype; it does not emit native machine code.

The complete pipeline



Each stage returns data to the next one. Errors stop the pipeline before query execution.

STAGE 1

Lexical analysis: characters become tokens

INPUT EXCERPT

```
SELECT name FROM Employee
WHERE salary > 80000;
```

TOKEN KINDS

```
SELECT IDENTIFIER FROM IDENTIFIER
WHERE IDENTIFIER OPERATOR NUMBER
SEMICOLON
```

Token classification

Keywords, identifiers, operators, literals, punctuation and EOF.

Diagnostic value

Tracks line / column information and catches invalid characters before parsing.

Demo evidence

Main query: 98 tokens recognized (excluding EOF).

Parsing: tokens become an abstract syntax tree

AST EXCERPT (SIMPLIFIED FOR READABILITY)

```
SELECT
  SELECT_ITEM AS department
    COLUMN d.name
  SELECT_ITEM AS employee_count
    FUNCTION COUNT
      COLUMN e.id
  SELECT_ITEM AS average_salary
    FUNCTION AVG
      COLUMN e.salary
FROM Employee AS e
  INNER JOIN Department AS d
  INNER JOIN Assignment AS a
  INNER JOIN Project AS p
WHERE ...
GROUP BY d.name
HAVING ...
ORDER BY ... DESC
LIMIT 5
```

Grammar and precedence

Recursive-descent parsing applies operator precedence and validates clause order.

AST purpose

Represents query structure without keeping every punctuation token as a node.

Fail-fast behavior

Malformed SELECT syntax is rejected before semantic analysis or execution.

STAGE 3

Semantic analysis: are names and references valid?

Alias map from demo

e → Employee
d → Department
a → Assignment

Successful query

All referenced tables, aliases and columns resolve; aggregate placement and grouping rules are checked for the supported subset.

DELIBERATELY INVALID QUERY

```
SELECT e.unknown_column
FROM Employee e;
```

SEMANTIC ANALYSIS FAILED
Unknown column: e.unknown_column

SQLite is not called after this failure.

STAGE 4

Relational IR: express the query as logical operations

LOGICAL PLAN EXCERPT

```
PROJECT [d.name, COUNT(e.id), AVG(e.salary)]
  LIMIT [5]
    SORT [AVG(e.salary) DESC]
      HAVING [(AVG(e.salary) > 65000)]
        AGGREGATE [d.name]
          FILTER [salary + budget predicates]
            JOIN [Employee ↔ Department]
              JOIN [Employee ↔ Assignment]
                JOIN [Assignment ↔ Project]
                  TABLE_SCAN [Employee AS e]
                  TABLE_SCAN [Department AS d]
                TABLE_SCAN [Assignment AS a]
              TABLE_SCAN [Project AS p]
```

Why not SQL text?

IR separates logical intent from the original clause formatting.

Useful nodes

Scan, join, filter, aggregate, HAVING filter, sort, limit and project.

For the viva

Explain the path from parsed clauses to relational operators.

STAGE 5

Cardinality: the optimizer estimates how many rows flow through operators

Relation	Synthetic rows	Used for
Employee	10,000	filter / joins
Department	4	joins / grouping
Project	1,200	budget predicate
Assignment	30,000	join size

Estimate

Predicted rows and relative work based on a simple synthetic schema.

Execution fixture

A much smaller seeded in-memory SQLite database, used to make the demo fast and repeatable.

Estimated row counts are not expected to match actual sample rows; they describe different dataset assumptions.

Plan alternatives: access paths and join algorithms

Table scan

Reads all rows in the estimated relation. Useful when a large fraction of rows is needed.

Index path estimate

Models a cheaper selective lookup when the schema has a matching modelled index.

Predicate pushdown

Estimates applying single-table filters before joins to reduce rows flowing upward.

Join method	Idea	COSTRA role
Nested loop	Compare rows pairwise	Relative model cost
Hash join	Build / probe hash table	Equality join alternative
Merge join	Process sorted inputs	Relative model cost

Inner-join orders are compared within a small cap; outer/cross-join order is preserved when arbitrary reordering may be unsafe.

Six candidate plans are compared using estimated cost

Numbers below are from the saved end-to-end demo; units are model units, not milliseconds.

Candidate	Estimated cost	Estimated rows
Plan 1	115,222.63	2.00
Plan 2	115,222.63	2.00
Plan 3	121,702.63	2.00
Plan 4	121,702.63	2.00
Plan 5	123,862.63	2.00
Plan 6	123,862.63	2.00

Selected minimum: 115,222.63 model units (Plan 1; Plan 2 ties at the same estimate).

What it means

- Compare alternatives
- Keep the lowest estimate
- Do not claim measured runtime

How COSTRA calculates estimated cost

N = input rows; L/R = left/right join rows; output rows are estimated, not measured.

ACCESS PATHS

FULL TABLE SCAN

Cost = N

INDEX SCAN

Cost = 2 × log2(max(2, N))
+ 0.60 × output_rows

PREDICATE

Cost = 0.20 × input_rows

Scan choice: compare full-scan + filter work against the index estimate; keep the lower estimate.

JOIN ALGORITHMS

NESTED-LOOP JOIN

Cost = 0.15 × L × max(1, R)

HASH JOIN

Cost = 1.10 × (L + R)
+ 0.25 × min(L, R)

MERGE JOIN

Cost = 0.80 × (L + R)
+ 0.45 × L × log2(max(2, L))
+ 0.45 × R × log2(max(2, R))

OTHER OPERATORS

AGGREGATE

Cost = 0.45 × input_rows

DISTINCT

Cost = 0.30 × input_rows

SORT

Cost = 0.35 × N × log2(max(2, N))

PROJECTION

Cost = 0.05 × output_rows

Operator costs accumulate up the plan tree. LIMIT reduces estimated output rows but adds no separate cost in this version.

Cost calculation from the current demo

These are the same synthetic row estimates shown in the terminal output.

1 / EMPLOYEE INDEX SCAN

Table rows N = 10,000
Estimated output = 7,200

Index cost
= 2 log2(10,000) + 0.60(7,200)
= 26.58 + 4,320
= 4,346.58

Full scan + predicate
= 10,000 + 0.20(10,000)
= 12,000.00

Decision: index path has the lower estimate.

2 / PROJECT INDEX SCAN

Table rows N = 1,200
Estimated output = 1,000

Index cost
= 2 log2(1,200) + 0.60(1,000)
= 20.46 + 600
= 620.46

Full scan + predicate
= 1,200 + 0.20(1,200)
= 1,440.00

Decision: index path has the lower estimate.

3 / HASH JOIN

Left rows L = 7,200
Right rows R = 30,000

Hash join cost
= 1.10(L + R) + 0.25 min(L, R)
= 1.10(37,200) + 0.25(7,200)
= 40,920 + 1,800
= 42,720.00

This is the join cost added at that step in the selected plan.

SELECTED PLAN TOTAL = 115,222.63 MODEL UNITS • Sum of the chosen operator/scan/join costs; the software keeps precision internally and displays rounded values.

The selected estimated plan applies selective paths early

SELECTED PLAN EXCERPT

```
PROJECT [d.name, COUNT(e.id), AVG(e.salary)]
  LIMIT [5]  {rows≈2, cost≈115222.53}
    SORT [AVG(e.salary) DESC]
      HAVING_FILTER [AVG(e.salary) > 65000]
        HASH_AGGREGATE [d.name]
          HASH_JOIN [Assignment ↔ Project]
            HASH_JOIN [Employee/Department ↔ Assignment]
              INDEX_SCAN Employee [salary > 60000]
              TABLE_SCAN Department
            INDEX_SCAN Project [budget > 100000]
```

Early filtering

Salary and budget predicates reduce rows before later operations.

Join ordering

Estimated join order changes intermediate work in the model.

Selection rule

Minimum estimated cost wins.

Keep COSTRA's estimate separate from SQLite's execution

COSTRA

- Builds logical IR
- Uses synthetic cardinality / selectivity
- Generates and ranks candidate estimates
- Displays selected estimated plan

SQLITE

- Receives normalized SQL
- Uses its own internal planner
- Executes against the seeded in-memory database
- Returns rows and its own EXPLAIN plan

COSTRA's chosen candidate is not injected into SQLite. This is a key implementation boundary, not a hidden feature.

The validated query is rendered as normalized SQL

SQL PRODUCED BY THE FORMATTER / PASSED TO SQLITE

```
SELECT d.name AS department, COUNT(e.id) AS employee_count, AVG(e.salary) AS average_salary
FROM Employee AS e
INNER JOIN Department AS d ON (e.dept_id = d.id)
INNER JOIN Assignment AS a ON (e.id = a.employee_id)
INNER JOIN Project AS p ON (a.project_id = p.id)
WHERE ((e.salary > 60000) AND (p.budget > 100000))
GROUP BY d.name
HAVING (AVG(e.salary) > 65000)
ORDER BY AVG(e.salary) DESC
LIMIT 5;
```

The normalized SQL carries the same supported logical clauses; SQLite then parses and plans it independently.

The demo returns three department rows

Department	Employee count	Average salary
Finance	1	91,000.0
Engineering	5	79,600.0
HR	1	69,000.0

Result count: 3 rows • Output is repeatable on the seeded in-memory fixture

Small fixture results are for demonstration and testing; they are not production-data performance claims.

SQLite exposes its own query plan separately

SQLITE EXPLAIN QUERY PLAN EXCERPT

```
SEARCH e USING INDEX idx_employee_salary (salary>?)
SEARCH a USING COVERING INDEX sqlite_autoindex_Assignment_1 (employee_id=?)
SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
SEARCH d USING INTEGER PRIMARY KEY (rowid=?)
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY
```

Why show this?

It is direct evidence of the execution engine's chosen strategy.

Compare carefully

This output is not COSTRA's selected estimated physical plan.

Separate labels prevent a false claim of plan injection.

Observed stage timings from the full demo

One local run on a tiny in-memory fixture; these timings are examples, not general benchmark claims.

Stage	Observed time	What it measures
Lexical analysis	0.093 ms	SQL text → tokens
Syntax analysis + AST	0.110 ms	Tokens → AST
Semantic analysis	0.029 ms	Reference validation
Cost estimation + plan search	0.907 ms	Synthetic plan ranking
SQLite execution	0.075 ms	Execute normalized SQL
SQLite EXPLAIN QUERY PLAN	0.036 ms	Inspect SQLite's plan

Repeatable benchmark

Separate script: 3 warm-ups and 30 measured repetitions across four query shapes.

Interpretation

Wall-clock timings are not the same as synthetic optimizer cost units.

Do not infer production performance from this single small-fixture run.

Invalid input stops at the stage that can explain it

Stage	Example	Expected behavior
Lexical	SELECT @ FROM Employee;	Illegal character reported with location
Syntax	SELECT FROM Employee;	Parser reports malformed SELECT
Semantic	SELECT e.unknown_column FROM Employee e;	Unknown column rejected before execution
Execution	Invalid AST bypassing semantic checks	SQLite error wrapped as ExecutionError

The goal is a readable diagnostic and a safe stop, not a Python traceback for expected bad SQL.

QUALITY ASSURANCE

52 automated regression tests pass in the checked run

52	6	3	4
tests passed	candidate plans	demo result rows	benchmark query shapes

Unit / component coverage

Lexer: token classes, comments, quoting and locations.
Parser: clauses, precedence and predicate syntax.
Semantics: aliases, tables, columns, aggregation and grouping.

Integration coverage

Optimizer: minimum-cost selection and join-order safety.
Execution: repeatable rows and SQLite plan.
CLI: complete demo, error demo and custom-query modes.

Re-run on the submission machine: python3 -m unittest -v

Each module has a clear responsibility

File	Role	Explain during viva
lexer.py	Characters → tokens	token kinds and source locations
parser.py / ast_nodes.py	SQL grammar → AST	recursive descent / precedence
semantic.py	Name and query checks	aliases, unknown/ambiguous columns
ir.py	Logical operator tree	scan, join, filter, aggregate, sort
schema.py / statistics.py	Synthetic metadata	row counts and selectivity
optimizer.py	Candidate estimates	paths, joins, cost ranking
executor.py / compiler.py	Orchestration + SQLite	fail-fast pipeline and result plan
main.py / test_costra.py	Demo CLI + tests	reproducible demo and regression checks

Run once, then choose what your professor asks to see

OPEN THE SWITCH-STYLE MENU

```
python3 main.py
```

MENU OPTIONS / 1–4

1. Review 1 — problem & design
2. Review 2 — core compiler modules
3. Review 3 — complete pipeline
4. Enter a custom SELECT query

MENU OPTIONS / 5–8

5. Cost formula + lowest-cost plan
6. Review 1 → Review 2 → Review 3
7. Semantic-error demonstration
8. Exit

DIRECT COMMAND-LINE SHORTCUTS

```
--review1-demo  |  --review2-demo
--cost-demo  |  --progress-demo
--demo  |  --error-demo  |  --query "SELECT ..."
```

Professor chooses

Enter a number from 1 to 8. After the walkthrough, COSTRA returns to the menu.

Cost and best plan

Option 5 prints the formulas, compares six candidates and identifies the minimum estimate.

Custom SQL

Option 4 accepts your SELECT statement at the SQL> prompt on one line.

RECOMMENDED LIVE DEMO FLOW

Choose the path that matches the professor’s question

01 · Review-wise summary

Choose option 6 to explain what changed from Review 1 to Review 3.

02 · Review 1 / Design

Choose option 1: problem, objectives, SQL scope and architecture.

03 · Review 2 / Core modules

Choose option 2: tokens, AST, semantics, relational IR and a sample result.

04 · Review 3 / Full demo

Choose option 3: full multi-join pipeline, plan selection and SQLite output.

05 · Professor’s own input

Choose option 4 and enter a SELECT query on one line.

06 · Cost calculation

Choose option 5 to compare formulas and explain why the minimum estimated cost wins.

FINAL MENU OPTIONS: 7 = semantic error demo • 8 = exit • Run python3 -m unittest -v to verify all 52 tests

The project is complete as an educational prototype, with clear scope limits

Current limitations

- SQL SELECT subset, not the full standard
- Synthetic statistics and heuristic costs
- Join enumeration is capped; not full dynamic programming
- Selected estimated plan is not injected into SQLite
- Benchmark dataset is a tiny in-memory fixture

Future enhancements

- Collect table and column statistics from data
- Add dynamic-programming / memoized join search
- Expand grouping/type semantics and SQL grammar
- Add an executor that truly follows COSTRA physical operators
- Benchmark on controlled, larger datasets

State limitations clearly; explaining a boundary is stronger than claiming a feature that is not implemented.

Questions the professor may ask

Question	Short answer to prepare
Why is this Compiler Design?	SQL is tokenized, parsed, semantically checked and transformed into a relational IR.
What is an AST?	A tree representation of the query's syntactic structure, without redundant punctuation.
What is cardinality?	The estimated number of rows produced by a relation or operator.
Are cost units runtime?	No; they are heuristic model units, not milliseconds.
Does SQLite run COSTRA's plan?	No; SQLite plans normalized SQL independently.
Why use an in-memory fixture?	Fast, repeatable outputs for demo/testing; not production benchmarking.

Thank You!

COSTRA — Cost-Based SQL Compiler & Query Optimizer

Arsh Shukla | 24BDS0113 | VIT Vellore

GitHub repository

github.com/arshshukla500newuser/COSTRA-SQL-Compiler